

Silvia Coffee Machine - Technical Manual

Table of Contents

1. [System Architecture](#)
2. [Hardware Integration](#)
3. [Software Architecture](#)
4. [Communication Protocol](#)
5. [State Machine](#)
6. [Safety Systems](#)
7. [Calibration Procedures](#)
8. [Troubleshooting Guide](#)
9. [Development Guide](#)
10. [API Reference](#)

System Architecture

Overview

The Silvia Coffee Machine system consists of two main components:

- **Arduino/Teensy Firmware:** Real-time hardware control and sensor management
- **PyQt6 Desktop Application:** User interface, data logging, and high-level control

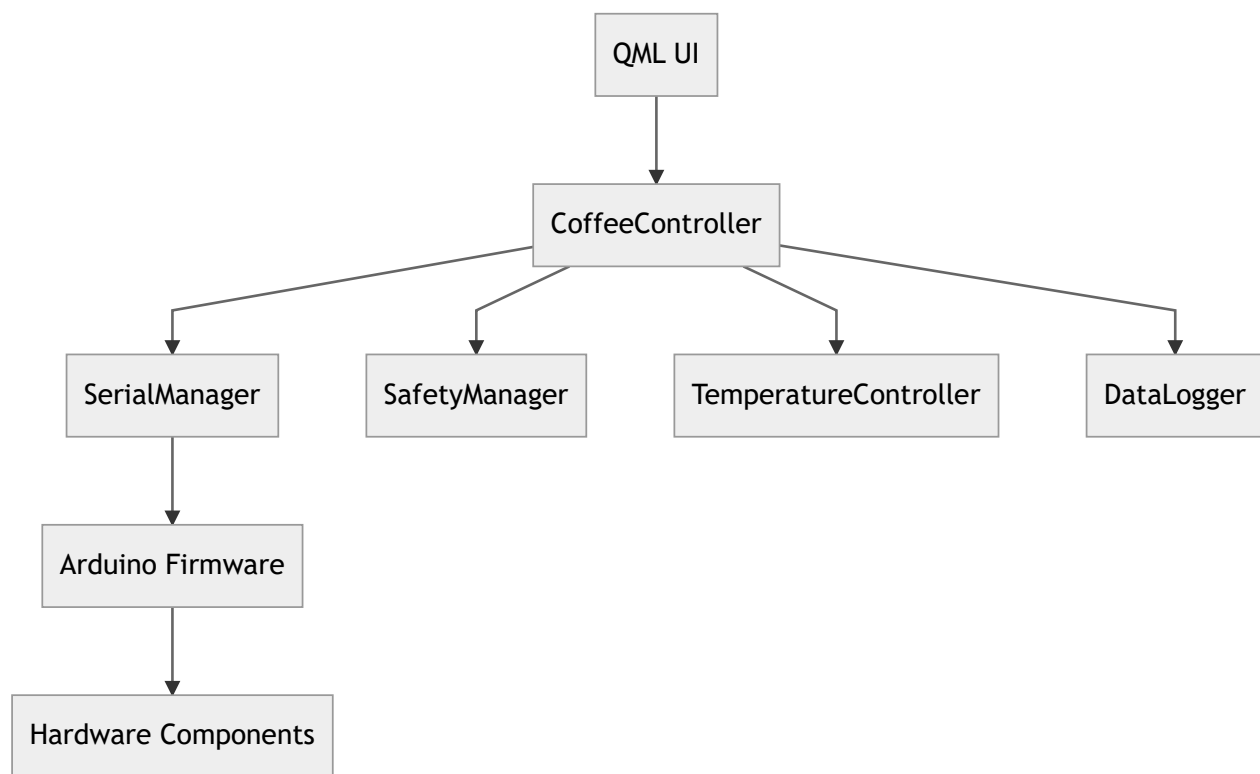
Data Flow

Sensors → Arduino → USB Serial → Python Backend → QML UI

↑ ↓

Hardware ← PWM Control ← Serial Commands ← User Input

Component Interaction



Hardware Integration

Sensor Integration

PT100 Temperature Sensor

- **Interface:** MAX31865 RTD-to-Digital Converter (SPI)

- **Configuration:** 3-wire PT100, 430Ω reference resistor
- **Resolution:** 0.03125°C per LSB
- **Update Rate:** 250ms
- **Fault Detection:** Automatic wire break and overvoltage detection

```
// Temperature reading with fault handling

uint16_t rtd = thermo.readRTD();

float temperature = thermo.temperature(RNOMINAL, RREF);

uint8_t fault = thermo.readFault();

if (fault) {

// Handle specific fault conditions

thermo.clearFault();

}
```

Pressure Sensor (Honeywell MIP)

- **Interface:** ADS1115 16-bit ADC (I2C)
- **Range:** 0-16 bar (configurable)
- **Resolution:** 16-bit (65536 levels)
- **Calibration:** Auto-calibration of zero pressure voltage
- **Update Rate:** 100ms

```
// Pressure measurement with calibration
```

```
adc.startSingleMeasurement();  
  
while (adc.isBusy()) { delay(1); }  
  
float voltage = adc.getResult_V();  
  
float pressure = mapPressure(voltage);
```

Dual Load Cell System

- **Interface:** Dual HX711 24-bit ADC
- **Configuration:** Two independent load cells summed
- **Resolution:** 24-bit (16.7M levels)
- **Calibration:** Individual calibration factors
- **Features:** Tare function, known-weight calibration

```
// Dual scale reading  
  
float weight0 = scale0.get_units(1);  
  
float weight1 = scale1.get_units(1);  
  
float totalWeight = weight0 + weight1;
```

Actuator Control

PWM Pump Control

- **Interface:** Direct PWM output (Pin 9)
- **Control:** Potentiometer input + software override
- **Range:** 0-255 PWM levels

- **Safety:** Minimum PWM threshold for motor startup

Heater Control

- **Interface:** SSR via PWM (Pin 15)
- **Control:** Multi-level PWM based on temperature difference
- **Safety:** Automatic shutoff on overheat
- **Efficiency:** Graduated power levels for precise control

Solenoid Valve

- **Interface:** Digital relay control (Pin 8)
- **States:** Open/Closed
- **Safety:** Fail-safe closed position
- **Timing:** Coordinated with pump operation

Software Architecture

PyQt6 Application Structure

Core Classes

CoffeeController (qml_backend.py)

- Central coordinator for all system operations
- QML signal/slot interface
- Serial communication management
- Safety system integration

SerialManager (serialcom/)

- Hardware abstraction layer
- Mock/Real hardware switching
- Command queuing and response handling
- Connection management

SafetyManager (safety_manager.py)

- Temperature monitoring
- Timeout protection
- Emergency stop coordination
- Warning system

TemperatureController (temperature_controller.py)

- PID-like temperature control
- Hysteresis management
- Target temperature tracking
- Heating state management

DataLogger (data_logger.py)

- Sensor data recording
- Brew session logging
- Error event tracking
- CSV export functionality

QML User Interface

Screen Hierarchy

ApplicationWindow

└─ StackView

| └─ HomeScreen (default)

| └─ BrewScreen

| └─ SteamScreen

| └─ SettingsScreen

└─ ConnectionStatus

└─ EmergencyStop

└─ ErrorDialog

└─ WarningDialog

Real-time Data Binding

```
// Temperature display with live updates
```

```
Text {
```

```
  text: window.currentTemp.toFixed(1) + "°C"
```

```
  color: window.currentTemp > targetTemp ? "#ff0000" : "white"
```

```
}
```

```
// Chart updates via Connections
```

```
Connections {  
  
  target: window  
  
  function onCurrentWeightChanged() {  
  
    coffeeChart.requestPaint()  
  
  }  
  
}
```

State Management

Application States

- **IDLE**: System ready, no active operations
- **HEATING_BREW**: Heating to brew temperature
- **HEATING_STEAM**: Heating to steam temperature
- **BREWING**: Active brewing with pump and valve
- **STEAMING**: Steam mode active
- **FLUSHING**: Cleaning cycle active

State Transitions

```
def startBrew(self):  
  
  if self.current_state == "IDLE":  
  
    self.current_state = "HEATING_BREW"  
  
    self.serial.send_command("START_BREW")
```

```
def beginBrew(self):  
  
    if self.current_state == "HEATING_BREW":  
  
        self.current_state = "BREWING"  
  
        self.serial.send_command("BEGIN_BREW")
```

Communication Protocol

Serial Protocol Specification

Command Format

COMMAND [PARAMETERS]\n

Response Format

STATUS:MESSAGE\n

DATA:field1,field2,...,fieldN\n

ERROR:description\n

Command Reference

Temperature Commands

SET_TEMP BREW 93.5 → OK:BREW_TEMP_SET

SET_TEMP STEAM 130.0 → OK:STEAM_TEMP_SET

Operation Commands

START_BREW → OK:BREW_STARTED

BEGIN_BREW → OK:BREWING_STARTED

STOP → OK:STOPPED

START_STEAM → OK:STEAM_STARTED

START_FLUSH → OK:FLUSH_STARTED

Calibration Commands

TARE_SCALES → OK:SCALES_TARED

CAL_SCALE 250.0 → OK:SCALES_CALIBRATED

SET_SCALE_CAL 420.1 421.4 → OK:SCALE_CAL_SET

System Commands

GET_STATUS → STATUS:state=1,temp=92.5,...

PING → PONG

ABORT → OK:ABORTED

Telemetry Data Format

```
DATA:state,temp,pressure,weight,pump%,valve,heater,timer,tared
```

```
DATA:3,92.5,8.25,25.3,85,1,1,15,1
```

Field Definitions:

- `state` : System state (0-5)
- `temp` : Current temperature (°C, 1 decimal)
- `pressure` : Current pressure (bar, 2 decimals)
- `weight` : Current weight (grams, 1 decimal)
- `pump%` : Pump power percentage (0-100)
- `valve` : Valve state (0=closed, 1=open)
- `heater` : Heater state (0=off, 1=on)
- `timer` : Brew timer (seconds)
- `tared` : Scales tared flag (0=no, 1=yes)

State Machine

Arduino State Machine

```
enum SystemState {  
  
STATE_IDLE = 0,  
  
STATE_HEATING_BREW = 1,  
  
STATE_HEATING_STEAM = 2,
```

```

STATE_BREWING = 3,

STATE_STEAMING = 4,

STATE_FLUSHING = 5

};

```

State Behavior Matrix

State	Heater	Pump	Valve	Conditions
IDLE	Brew Temp	OFF	Closed	Default state
HEATING_BREW	Brew Temp	OFF	Closed	Heating to brew temp
HEATING_STEAM	Steam Temp	OFF	Closed	Heating to steam temp
BREWING	Brew Temp	Variable	Open	Active brewing
STEAMING	Steam Temp	OFF	Open	Steam wand active
FLUSHING	OFF	Full	Open	Cleaning cycle

Transition Conditions

```

void updateSystemLogic() {

switch (sys.state) {

case STATE_HEATING_BREW:

controlHeater(sys.brewTemp);

if (tempReached && userCommand == "BEGIN_BREW") {

```

```
    sys.state = STATE_BREWING;

}

break;

case STATE_BREWING:

    controlHeater(sys.brewTemp);

    controlPump(true);

    setValve(true);

    break;

}

}
```

Safety Systems

Multi-Layer Safety Architecture

Hardware Safety (Arduino)

1. **Watchdog Timer:** System reset on firmware hang
2. **Temperature Limits:** Hard-coded maximum temperatures
3. **Fail-Safe States:** Safe defaults on power-up
4. **Input Validation:** Parameter range checking

Software Safety (Python)

1. **Communication Timeout:** Emergency stop on data loss

2. **Temperature Monitoring:** Continuous overheat detection
3. **Operation Timeouts:** Maximum brew/steam time limits
4. **User Emergency Stop:** Manual safety override

Safety Manager Implementation

```
class SafetyManager(QObject):  
  
    def check_temperature(self, temp):  
  
        if temp > self.max_temp:  
  
            self.emergencyStop.emit(f"OVERHEAT: {temp}°C")  
  
        return False  
  
        return True  
  
    def _safety_check(self):  
  
        # Communication timeout  
  
        if time.time() - self.last_data_time > self.comm_timeout:  
  
            self.emergencyStop.emit("Communication timeout")  
  
        # Operation timeouts  
  
        if self.brew_start_time:  
  
            if time.time() - self.brew_start_time > self.max_brew_time:  
  
                self.emergencyStop.emit("Brew timeout")
```

Emergency Stop Sequence

1. **Trigger Detection:** Timeout, overheat, or user command

2. **Immediate Actions:**

- Send ABORT command to Arduino
- Stop all Python timers
- Set UI to safe state

3. **Hardware Response:**

- Turn off heater (PWM = 0)
- Stop pump (PWM = 0)
- Close valve (Digital LOW)
- Reset state to IDLE

Calibration Procedures

Temperature Sensor Calibration

The PT100 sensor is factory calibrated, but verification is recommended:

1. **Ice Bath Test** (0°C reference)

- Submerge sensor in ice water
- Reading should be 0.0°C $\pm$ 0.5°C
- Adjust RNOMINAL if necessary

2. **Boiling Water Test** (100°C reference)

- Submerge sensor in boiling water at sea level
- Reading should be 100.0°C $\pm$ 0.5°C

- Check for linear response

Pressure Sensor Calibration

Automatic zero-point calibration on startup:

```
// Auto-calibrate zero pressure

float sumVoltage = 0;

for (int i = 0; i < 50; i++) {

    sumVoltage += adc.getResult_V();

    delay(50);

}

calibratedVZero = sumVoltage / 50;
```

Manual calibration with known pressure source:

1. Apply known pressure (e.g., 9 bar)
2. Record voltage reading
3. Calculate scaling factor
4. Update V_MAX constant

Scale Calibration Procedure

1. Tare Scales

```
controller.tareScales() # Zero the scales
```

2. Place Known Weight

- Use certified calibration weight (100g-500g recommended)
- Ensure stable placement

3. Calibrate

```
controller.calibrateScales(250.0) # 250g calibration weight
```

4. Verification

- Remove weight, verify zero reading
- Place different known weights to verify linearity

Calibration Data Storage

```
{  
  
  "brew_temp": 93.0,  
  
  "steam_temp": 127.0,  
  
  "scale_cal_0": 420.0983,  
  
  "scale_cal_1": 421.365  
}
```

Troubleshooting Guide

Hardware Issues

Temperature Sensor Problems

Symptom: Erratic temperature readings

Causes:

- Loose PT100 connections
- MAX31865 wiring issues
- Electrical interference

Diagnosis:

```
uint8_t fault = thermo.readFault();

if (fault & MAX31865_FAULT_HIGHTHRESH) {

  Serial.println("RTD High Threshold");

}

// Check other fault bits...
```

Solutions:

- Verify 3-wire PT100 connection
- Check SPI wiring (CS, MOSI, MISO, CLK)
- Add ferrite cores for EMI suppression
- Ensure proper grounding

Pressure Sensor Issues

Symptom: Incorrect pressure readings

Causes:

- I2C communication failure
- Incorrect zero calibration
- Sensor drift

Diagnosis:

```
if (!adc.init()) {  
  
  Serial.println("ADS1115 initialization failed");  
  
}
```

Solutions:

- Check I2C wiring (SDA, SCL, VCC, GND)
- Verify I2C address (0x48 default)
- Recalibrate zero pressure point
- Check for air leaks in pressure system

Scale Problems

Symptom: Unstable weight readings

Causes:

- Mechanical vibration
- Poor HX711 connections
- Load cell damage

Solutions:

- Isolate scales from vibration

- Check HX711 wiring and power supply
- Verify load cell mounting
- Recalibrate with known weights

Software Issues

Communication Problems

Symptom: "DISCONNECTED" status

Causes:

- Wrong serial port
- Incorrect baud rate
- USB cable issues
- Arduino not programmed

Diagnosis:

```
# Check available ports

import serial.tools.list_ports

ports = serial.tools.list_ports.comports()

for port in ports:

    print(f"Port: {port.device}")
```

Solutions:

- Verify correct COM port in config

- Check USB cable and connections
- Re-upload Arduino firmware
- Try different baud rates

UI Responsiveness

Symptom: Slow or frozen interface

Causes:

- High CPU usage
- Memory leaks
- Qt threading issues

Solutions:

- Check system resources
- Restart application
- Update graphics drivers
- Reduce telemetry frequency

Error Code Reference

Error Code	Description	Action
-----	-----	-----
PT100_FAULT	Temperature sensor fault	Check wiring, clear fault
ADS1115_INIT_FAILED	Pressure sensor init failed	Check I2C connections
COMM_TIMEOUT	Communication lost	Check USB connection

| OVERHEAT | Temperature too high | Emergency stop, check heater |

| BREW_TIMEOUT | Brew time exceeded | Stop brewing, check system |

| STEAM_TIMEOUT | Steam time exceeded | Stop steaming, check system |

Development Guide

Setting Up Development Environment

Python Environment

```
# Create virtual environment

python -m venv silvia_env

source silvia_env/bin/activate # Linux/Mac

silvia_env\Scripts\activate # Windows
```

```
# Install dependencies

pip install PyQt6 pyserial
```

Arduino Environment

```
# Install required libraries via Arduino Library Manager

- Adafruit MAX31865 library

- ADS1115_WE library
```

- HX711 library

Mock Mode Development

For UI development without hardware:

```
# Enable mock mode in config.py
```

```
USE MOCK_SERIAL = True
```

```
# Or use command line
```

```
python run_silvia.py --mock
```

Mock mode provides:

- Simulated sensor readings
- Realistic temperature curves
- Command acknowledgments
- State transitions

Adding New Features

Hardware Integration

1. Update Arduino firmware

- Add sensor reading code
- Update telemetry format

- Add new commands

2. Update communication protocol

- Define new commands
- Update data format
- Add error handling

3. Update Python backend

- Parse new telemetry data
- Add command methods
- Update safety checks

UI Development

1. QML Interface

- Add new screens/components
- Update data bindings
- Add user controls

2. Backend Integration

- Connect QML signals
- Update data models
- Add validation

Testing Procedures

Unit Testing

```
# Test serial communication

def test_serial_commands():

    serial_mgr = MockSerialManager()

    serial_mgr.send_command("START_BREW")

    assert serial_mgr.last_command == "START_BREW"


# Test safety systems

def test_temperature_safety():

    safety = SafetyManager()

    result = safety.check_temperature(170.0) # Over limit

    assert result == False
```

Integration Testing

1. **Hardware-in-Loop:** Test with real sensors
2. **End-to-End:** Full brewing cycle testing
3. **Safety Testing:** Emergency stop scenarios
4. **Performance Testing:** Response time measurement

Code Style Guidelines

Python (PEP 8)

```
class CoffeeController(QObject):
```

```
"""Main controller for coffee machine operations."""

def __init__(self, parent=None):

    super().__init__(parent)

    self.current_temp = 0.0

    def start_brew(self):

        """Start the brewing process."""

        if self.connected:

            self.serial.send_command("START_BREW")
```

C++ (Arduino Style)

```
void updateSensors() {

    // Read temperature with error handling

    if (millis() - lastTempRead >= TEMP_READ_INTERVAL) {

        sys.currentTemp = thermo.temperature(RNOMINAL, RREF);

        lastTempRead = millis();

    }

}
```

QML (Qt Conventions)

```
Rectangle {
```

```
id: temperatureDisplay

width: 200

height: 100

color: "#34495e"

Text {

anchors.centerIn: parent

text: currentTemp.toFixed(1) + "°C"

color: "white"

font.pixelSize: 24

}

}
```

API Reference

CoffeeController Methods

Temperature Control

```
@pyqtSlot(float, float)

def setTemperatures(self, brew_temp, steam_temp):

    """Set target temperatures for brew and steam modes."""

    @pyqtSlot()

    def startBrew(self):
```

```
"""Start heating to brew temperature."""
```

```
@pyqtSlot()
```

```
def beginBrew(self):
```

```
"""Begin actual brewing process."""
```

Operation Control

```
@pyqtSlot()
```

```
def startSteam(self):
```

```
"""Start heating to steam temperature."""
```

```
@pyqtSlot()
```

```
def startFlush(self):
```

```
"""Start flush cleaning cycle."""
```

```
@pyqtSlot()
```

```
def stopBrew(self):
```

```
"""Stop current brewing operation."""
```

Scale Management

```
@pyqtSlot()
```

```
def tareScales(self):
```

```
"""Zero the scales."""
```

```
@pyqtSlot(float)

def  calibrateScales(self, knownWeight):

    """Calibrate scales with known weight."""
```

Safety

```
@pyqtSlot()

def  emergencyStop(self):

    """Manual emergency stop."""
```

Signals

Data Updates

```
temperatureChanged = pyqtSignal(float)

pressureChanged = pyqtSignal(float)

weightChanged = pyqtSignal(float)

stateChanged = pyqtSignal(str)
```

Status Updates

```
connectionStatusChanged = pyqtSignal(bool)

errorOccurred = pyqtSignal(str)
```

```
warningIssued = pyqtSignal(str)
```

Arduino Functions

Core Functions

```
void processSerialCommands(); // Handle incoming commands
```

```
void updateSensors(); // Read all sensors
```

```
void updateSystemLogic(); // State machine logic
```

```
void sendTelemetry(); // Send data to PC
```

Control Functions

```
void controlHeater(float targetTemp); // Temperature control
```

```
void controlPump(bool enable); // Pump control
```

```
void setValve(bool open); // Valve control
```

Utility Functions

```
void tareScales(); // Zero scales
```

```
void calibrateScales(float weight); // Calibrate scales
```

```
float mapPressure(float voltage); // Convert voltage to pressure
```

```
void stopCurrentOperation(); // Emergency stop
```

This technical manual provides comprehensive information for understanding, maintaining, and extending the Silvia Coffee Machine control system. For additional support, refer to the troubleshooting section or contact the development team.